

Лабораторная работа №1
по дисциплине «Инжиниринг данных»
Вариант №2

ЦЕЛЬ РАБОТЫ

Отработка навыков использования библиотек *NumPy*, *Pandas*, *Matplotlib*.

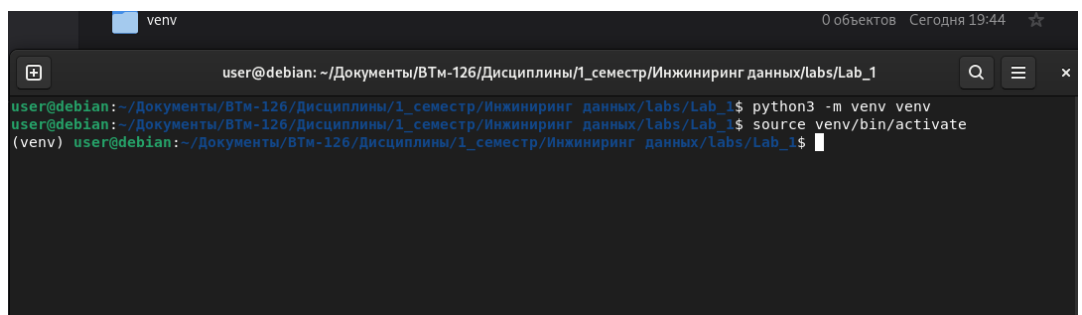
ВЫПОЛНЕНИЕ РАБОТЫ

1 Этап: Создание проекта

Состав работ включает в себя:

- Подготовка виртуального окружения
- Создание файла *main.py*
- Установка необходимых зависимостей для работы проекта

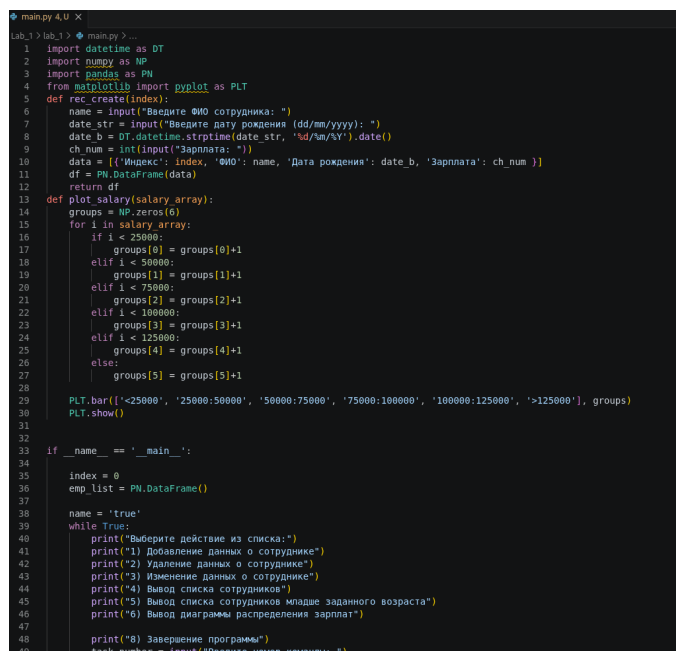
Для подготовки виртуального окружения (*venv*) в *Python* в терминале была выполнена команда: *python3 -m venv venv*



```
venv
0 объектов  Сегодня 19:44 ☆
user@debian: ~/Документы/ВТМ-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1
user@debian: ~/Документы/ВТМ-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ python3 -m venv venv
user@debian: ~/Документы/ВТМ-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ source venv/bin/activate
(venv) user@debian: ~/Документы/ВТМ-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$
```

Рис.1 Создание виртуального окружения *venv*

В папке проекта был создан файл *main.py* с начальным кодом из методических указаний



```
main.py 4,0 X
Lab_1 > Lab_1 > main.py > ...
1 import datetime as DT
2 import numpy as NP
3 import pandas as PN
4 from matplotlib import pyplot as PLT
5
6 def rec_create(index):
7     name = input("Введите ФИО сотрудника: ")
8     date_str = input("Введите дату рождения (dd/mm/yyyy): ")
9     date_b = DT.datetime.strptime(date_str, '%d/%m/%Y').date()
10    ch_num = int(input("Зарплата: "))
11    data = {'Индекс': index, 'ФИО': name, 'Дата рождения': date_b, 'Зарплата': ch_num}
12    df = PN.DataFrame(data)
13    return df
14
15 def plot_salary(salary_array):
16    groups = NP.zeros(6)
17    for i in salary_array:
18        if i < 25000:
19            groups[0] = groups[0]+1
20        elif i < 50000:
21            groups[1] = groups[1]+1
22        elif i < 75000:
23            groups[2] = groups[2]+1
24        elif i < 100000:
25            groups[3] = groups[3]+1
26        elif i < 125000:
27            groups[4] = groups[4]+1
28        else:
29            groups[5] = groups[5]+1
30
31    PLT.bar(['<25000', '25000:50000', '50000:75000', '75000:100000', '100000:125000', '>125000'], groups)
32    PLT.show()
33
34 if __name__ == '__main__':
35     index = 0
36     emp_list = PN.DataFrame()
37     name = 'True'
38     while True:
39         print("Выберите действие из списка:")
40         print("(1) Добавление данных о сотруднике")
41         print("(2) Удаление данных о сотруднике")
42         print("(3) Изменение данных о сотруднике")
43         print("(4) Вывод списка сотрудников")
44         print("(5) Вывод списка сотрудников младше заданного возраста")
45         print("(6) Вывод диаграммы распределения зарплат")
46         print("(0) Завершение программы")
47         task_number = input("Введите номер команды: ")
```

Рис. 2 Файл *main.py* в изначальном виде

В соответствии с задействованными библиотеками в изначальном коде *main.py*, в виртуальном окружении *venv* были установлены пакеты *numpy*, *pandas*, *matplotlib*

```
(venv) user@debian: ~/Документы/ВТМ-126/Дисциплины/1 семестр/Инжиниринг данных/Labs/Lab_1 $ pip install numpy pandas matplotlib
Collecting numpy
  Downloading numpy-2.5.2-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting pandas
  Downloading pandas-3.0.5-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting matplotlib
  Using cached matplotlib-3.11.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (80 kB)
  Using cached python-dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting contourpy==1.0.1 (from matplotlib)
  Using cached contourpy-1.3.3-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
Collecting cycler==0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools==4.28.2 (from matplotlib)
  Downloading fonttools-4.64.0-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (123 kB)
Collecting kiwisolver==1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.2 kB)
Collecting packaging==20.0 (from matplotlib)
  Using cached packaging-20.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pillow==9 (from matplotlib)
  Using cached pillow-12.3.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Collecting pyparsing==3 (from matplotlib)
  Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting six==1.5 (from python-dateutil>=2.8.2->pandas)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Download numpy-2.5.2-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7 MB)
----- 15.7/16.7 MB 0.4 MB/s eta 0:00:00
Download pandas-3.0.5-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (10.9 MB)
----- 10.9/10.9 MB 0.7 MB/s eta 0:00:00
Using cached matplotlib-3.11.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (10.0 MB)
Using cached contourpy-1.3.3-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (362 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Download fonttools-4.64.0-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.4 MB)
----- 5.4/5.4 MB 1.4 MB/s eta 0:00:00
Download kiwisolver-1.5.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5 MB)
----- 1.5/1.5 MB 6.4 MB/s eta 0:00:00
Using cached packaging-20.3-py3-none-any.whl (129 kB)
Using cached pillow-12.3.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.9 MB)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached python-dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, pyparsing, pillow, packaging, numpy, kiwisolver, fonttools, cycler, python-dateutil, contourpy, pandas, matplotlib
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.64.0 kiwisolver-1.5.1 matplotlib-3.11.1 numpy-2.5.2 packaging-20.3 pandas-3.0.5 pillow-12.3.0 pyparsing-3.3.2 python-dateutil-2.9.0.post0 six-1.17.0
(venv) user@debian: ~/Документы/ВТМ-126/Дисциплины/1 семестр/Инжиниринг данных/Labs/Lab_1 $
```

Рис.3 Установка основных зависимостей проекта

При тестовом запуске *main.py* при выводе графической диаграммы высвывается предупреждение «*UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown*». Данное сообщение возникает из-за того, что *Matplotlib* в данный момент использует бэкенд, не поддерживающий графический интерфейс (*Agg*).

```
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
8) Завершение программы
Введите номер команды: 6
/home/user/Документы/ВТМ-126/Дисциплины/1 семестр/Инжиниринг данных/Labs/Lab_1/main.py:44: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
Выберите действие из списка:
```

Рис.4 Ошибка *matplotlib*

Для устранения данной ошибки необходимо в виртуальное окружение *venv* дополнительно установить пакет графического интерфейса — *pyqt6*

Таким образом, проект для выполнения задания готов; дополнительно в файл *requirements.txt* были сохранены пакеты виртуального окружения *venv* и их версии.

```
(venv) user@debian:~/Документы/ВТм-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ pip install pyqt6
Collecting pyqt6
  Using cached pyqt6-6.11.0-cp310-abi3-manylinux_2_34_x86_64.whl.metadata (2.1 kB)
Requirement already satisfied: PyQt6-sip<14,>=13.8 in ./venv/lib/python3.13/site-packages (from pyqt6) (13.12.0)
Requirement already satisfied: PyQt6-Qt6<6.12.0,>=6.11.0 in ./venv/lib/python3.13/site-packages (from pyqt6) (6.11.2)
Using cached pyqt6-6.11.0-cp310-abi3-manylinux_2_34_x86_64.whl (8.3 MB)
Installing collected packages: pyqt6
Successfully installed pyqt6-6.11.0
(venv) user@debian:~/Документы/ВТм-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ pip freeze > requirements.txt
(venv) user@debian:~/Документы/ВТм-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ cat requirements.txt
contourpy==1.3.3
cycler==0.12.1
fonttools==4.64.0
kiwisolver==1.5.1
matplotlib==3.11.1
numpy==2.5.2
packaging==26.3
pandas==3.0.5
pillow==12.3.0
pyparsing==3.3.2
PyQt5-Qt5==5.15.19
PyQt5_sip==12.19.0
PyQt6==6.11.0
PyQt6-Qt6==6.11.2
PyQt6_sip==13.12.0
python-dateutil==2.9.0.post0
six==1.17.0
(venv) user@debian:~/Документы/ВТм-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$
```

Рис.5 Установка *pyqt6* и вывод всех пакетов в файл

Этап 2. Выполнение заданий

Состав работ включает в себя:

- Обоснование архитектуры проекта
- Разработка кода проекта
- Тестирование кода проекта

Чтобы упростить создание и поддержку кода, было принято решение осуществить переход на объектно-ориентированное программирование

В рамках предметной области существует сущность Сотрудник, которая содержит индекс, ФИО, дата рождения, текущая зарплата, а также (в соответствии с вариантом №2) семейное положение.

Для хранения данных сущности Сотрудник был выделен отдельный *@dataclass Worker*: ввод данных осуществляется при вызове *@classmethod from_input*. Данный метод предназначен для работы с двумя сценариями:

- Добавление данных — сценарий по умолчанию, в метод не передается параметр *old_data*, заполнение данных сущности осуществляется вводом с клавиатуры
- Обновление данных — дополнительный сценарий, в метод передается параметр *old_data*, если пользователь не заполняет данные вводом с

клавиатуры(нажимает *Enter*), то в атрибуты подгружаются данные из *old_data*

```
worker_factory.py 1 worker.py M X worker_controller.py 1
Lab_1 > lab_1 > src > worker.py > Worker > from_input
1 import datetime as DT
2 from dataclasses import dataclass, field
3
4 @dataclass
5 class Worker:
6     name: str
7     date_b: DT.date
8     ch_num: int
9     isRinged: bool
10
11     data: dict = field(init=False)
12
13     def __post_init__(self):
14         self.data = {'ФИО': self.name, 'Дата рождения': self.date_b, 'Зарплата': self.ch_num, 'Женат/замужем': self.isRinged }
15
16
17 @classmethod
18 def from_input(cls, old_data=None):
19     name = input("Введите ФИО сотрудника: ") or (None if old_data is None else old_data.get("ФИО"))
20     date_str = input("Введите дату рождения (dd/mm/yyyy): ") or (None if old_data is None else old_data.get("Дата рождения").strftime('%d/%m/%Y'))
21     ch_num = (input("Зарплата: ") or 0) if old_data is None else old_data.get("Зарплата")
22     ring_str = (input("Семейное положение (женат/замужем/холост): ").strip().lower()) or (False if old_data is None else str(old_data.get("Женат/замужем")))
23
24     date_b = DT.datetime.strptime(date_str, '%d/%m/%Y')
25     isRinged = ring_str in ['женат', 'замужем', 'true', '1', 'True']
26
27
28     return cls(name=name, date_b=date_b, ch_num=int(ch_num), isRinged=isRinged)
```

Рис.6 Листинг класса *Worker*

Для удобного создания экземпляров класса *Worker* был создан отдельный класс *WorkerFactory*. Данный класс при инициализации создает датафрейм *df(PN.DataFrame)* с предустановленными колонками

Непосредственно создание сущности *Worker* и сохранение в датафрейм осуществляется через метод *create*. Данный метод предназначен для работы с двумя сценариями:

- Добавление данных с помощью клавиатуры— сценарий по умолчанию, в метод не передается параметр *w*, заполнение данных сущности осуществляется вводом с клавиатуры
- Добавление данных программно— дополнительный сценарий, в метод передается параметр *w* — экземпляр *Worker*. Данный объект будет генерироваться вне класса — например программно при тестировании класса или в классе работы с файлом(чтение)

```

Lab_1 > lab_1 > src > worker_factory.py > WorkerFactory > __str__
1 import pandas as PN
2 from .worker import Worker
3
4 class WorkerFactory:
5     def __init__(self):
6         self.df = PN.DataFrame(columns=['ФИО', 'Дата рождения', 'Зарплата', 'Женат/замужем'])
7
8     def create(self, w: Worker=None):
9         w = Worker.from_input() if w is None else w
10        new_row = PN.DataFrame([w.data])
11        self.df = PN.concat([self.df, new_row], ignore_index=True)
12        self.df.index = PN.RangeIndex(start=1, stop=1+len(self.df), name='Индекс')
13        self.df['Дата рождения'] = PN.to_datetime(self.df['Дата рождения'])
14
15
16    def __str__(self):
17        return self.df.to_string()

```

Рис.7 Листинг класса *WorkerFactory*

Для удаления и добавления данных класс *WorkerFactory* был расширен и выделен в отдельный класс *FeatureWorkerFactory* с соответствующими методами

```

worker.py  main.py  feature_worker_factory.py X
Lab_1 > lab_1 > src > feature_worker_factory.py > ...
1 from .worker_factory import WorkerFactory
2 from .decorator import Decorator
3 from .worker import Worker
4
5 class FeatureWorkerFactory(WorkerFactory):
6     def __init__(self):
7         super().__init__()
8
9     @Decorator.inputId
10    def delete(self, id: int=None):
11        self.df.drop(index=id, inplace=True, errors='ignore')
12
13    @Decorator.inputId
14    def update(self, id: int=None):
15        old_data = self.df.loc[id]
16        print("Введите новые данные; для сохранения текущих значений просто нажмите Enter ")
17        w = Worker.from_input(old_data)
18        new_data = w.data
19
20        result = {k: new_data[k] if new_data[k] not in (None, '') else old_data.get(k) for k in new_data}
21        self.df.loc[id] = result

```

Рис.8 Листинг класса *FeatureWorkerFactory*

Для обеспечения чистоты кода класса *FeatureWorkerFactory* был выделен отдельный класс для проверки ввода идентификатора сущности(id) для методов *delete* и *update*

```

worker.py  main.py  decorator.py X
Lab_1 > lab_1 > src > decorator.py > ...
1  from functools import wraps
2
3  class Decorator:
4      @staticmethod
5      def inputId(func):
6          @wraps(func)
7          def wrapper(self, *args, **kwargs):
8              result=None
9              value=(input("Введите id:")).rstrip()
10
11              if value.isdigit():
12                  kwargs['id'] = int(value)
13                  result = func(self, *args, **kwargs)
14              else:
15                  print("Неправильный формат id")
16              return result
17          return wrapper

```

Рис.9 Листинг класса *Decorator*

Генерация графиков (график распределения зарплат; круговая диаграмма, содержащая информацию о количестве сотрудников в браке и вне брака) выделена в отдельный класс *GraphController*

```

worker.py  main.py  graph_controller.py 3 X
Lab_1 > lab_1 > src > graph_controller.py > ...
1  import numpy as NP
2  from matplotlib import pyplot as PLT
3
4  class GraphController:
5      def plot_salary(self, salary_array):
6          groups = NP.zeros(6)
7          for i in salary_array:
8              if i < 25000:
9                  groups[0] = groups[0]+1
10             elif i < 50000:
11                 groups[1] = groups[1]+1
12             elif i < 75000:
13                 groups[2] = groups[2]+1
14             elif i < 100000:
15                 groups[3] = groups[3]+1
16             elif i < 125000:
17                 groups[4] = groups[4]+1
18             else:
19                 groups[5] = groups[5]+1
20
21         PLT.bar(['<25000', '25000:50000', '50000:75000', '75000:100000', '100000:125000', '>125000'], groups)
22         PLT.show()
23
24     def plot_circle(self, ring_array):
25         labels = ["Холост", "Женат/Замужем"]
26         groups = NP.zeros(2)
27         for el in ring_array:
28             i=0 if el is True else 1
29             groups[i]+=1
30         PLT.pie(groups, labels=labels, autopct='%1.1f%%')
31         PLT.show()

```

Рис.10 Листинг класса *GraphController*

В качестве центрального связующего звена программы был выделен класс *WorkerController* с соответствующими методами:

- Вывод меню программы — метод *print_task_list*
- Добавление данных о сотруднике — метод *create*
- Удаление данных о сотруднике — метод *delete*

- Изменение данных о сотруднике — метод *update*
- Вывод списка сотрудников — метод *selectAll*
- Вывод списка сотрудников младше заданного возраста — метод *selectByQuery*
- Вывод диаграммы распределения зарплат — метод *plotBar*
- Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака — метод *plotCircle*
- Завершение программы — метод *exit*

```

worker_controller.py 1 X
Lab_1 > lab_1 > src > worker_controller.py > WorkerController > print_task_list
1  import pandas as PN
2  from .feature_worker_factory import FeatureWorkerFactory
3  from .graph_controller import GraphController
4
5  class WorkerController:
6      def __init__(self):
7          self.__factory= FeatureWorkerFactory()
8          self.__graph= GraphController()
9          self.active=True
10
11      def create(self):
12          """Добавление данных о сотруднике"""
13          self.__factory.create()
14
15      def delete(self):
16          """Удаление данных о сотруднике"""
17          self.__factory.delete()
18
19      def update(self):
20          """Изменение данных о сотруднике"""
21          self.__factory.update()
22
23      def selectAll(self):
24          """Вывод списка сотрудников"""
25          result=self.__factory
26          print(result)
27
28
29      def selectByQuery(self):
30          """Вывод списка сотрудников младше заданного возраста"""
31          df=self.__factory.df
32          now = PN.Timestamp.now()
33
34          query=int(input("Введите возраст"))
35
36          result = df[(now.year - df["Дата рождения"].dt.year) < query]
37          print(result)
38
39
40      def plotBar(self):
41          """Вывод диаграммы распределения зарплат"""
42          data=self.__factory.df['Зарплата'].to_numpy()
43          self.__graph.plot_salary(data)
44
45      def plotCircle(self):
46          """Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака. """
47          data=self.__factory.df['Женат/замужем'].to_numpy()
48          self.__graph.plot_circle(data)
49
50      def exit(self):
51          """Завершение программы"""
52          self.active=False
53
54      def print_task_list(self):
55          print(""" Выберите действие из списка:
56          1) {self.create.__doc__}
57          2) {self.delete.__doc__}
58          3) {self.update.__doc__}
59          4) {self.selectAll.__doc__}
60          5) {self.selectByQuery.__doc__}
61          6) {self.plotBar.__doc__}
62          7) {self.plotCircle.__doc__}
63          8) {self.exit.__doc__}
64          """)

```

Рис.11 Листинг класса *WorkerController*

Классы были объединены в отдельный пакет lab_1

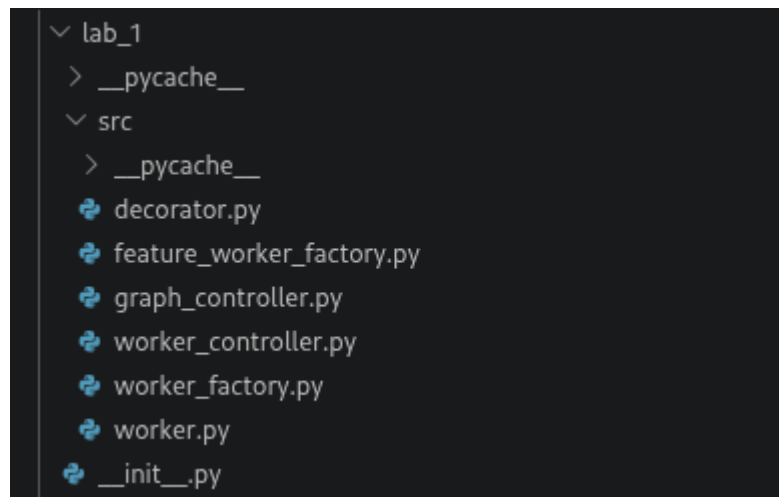


Рис.12 Физическая структура пакета lab_1

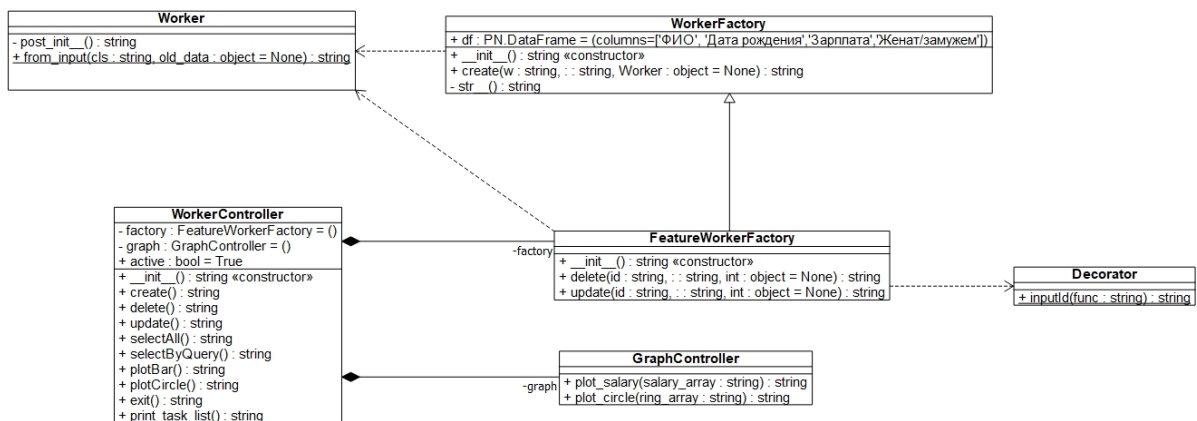


Рис.13 Схема взаимодействия классов в Umbrello5

```

worker.py  main.py  __init__.py X
Lab_1 > lab_1 > __init__.py
1  from .src.worker_factory import WorkerFactory
2  from .src.decorator import Decorator
3  from .src.feature_worker_factory import FeatureWorkerFactory
4  from .src.graph_controller import GraphController
5  from .src.worker_controller import WorkerController
6  from .src.worker import Worker
7

```

Рис.14 Листинг файла пакета lab_1 (__init__.py)

```

1  from lab_1 import WorkerController
2
3  if __name__ == '__main__':
4
5      c=WorkerController()
6      name = 'true'
7      while c.active:
8          c.print_task_list()
9          task_number = input("Введите номер команды: ")
10
11         match task_number:
12             case '1':
13                 c.create()
14             case '2':
15                 c.delete()
16             case '3':
17                 c.update()
18             case '4':
19                 c.selectAll()
20             case '5':
21                 c.selectByQuery()
22             case '6':
23                 c.plotBar()
24             case '7':
25                 c.plotCircle()
26             case '8':
27                 c.exit()
28             case _:
29                 print(f"Неизвестный номер команды: {task_number}")
30

```

Рис.15 Листинг файла *main.py*

Для проверки работоспособности программы было осуществлено ручное тестирование по следующему сценарию:

- Запуск программы — результат отражен на рис. 16
- Ввод пятерых сотрудников и вывод списка на экран — результат отражен на рис. 17
- Удаление сотрудника с идентификатором 2; вывод списка оставшихся сотрудников — результат отражен на рис. 18
- Обновление данных сотрудника с идентификатором 1 — полная замена данных; вывод списка на экран — результат отражен на рис. 19
- Частичное обновление данных сотрудника с идентификатором 3 — изменение зарплаты — результат отражен на рис. 20
- Вывод списка сотрудников младше 25 — результат отражен на рис. 21
- Построение графиков — результат отражен на рис. 22-23
- Выход из программы — результат отражен на рис. 24

```

venv) user@debian: ~/Документы/ВТМ-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ python3 main.py
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака.
8) Завершение программы

Введите номер команды:

```

Рис.16 Начальный экран программы

```

4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака.
8) Завершение программы

Введите номер команды: 1
Введите ФИО сотрудника: Фролов Кирилл Петрович
Введите дату рождения (dd/mm/yyyy): 1/1/1995
Зарплата: 65000
Семейное положение (женат/замужем/холост): 1
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака.
8) Завершение программы

Введите номер команды: 1
Введите ФИО сотрудника: Меньшова Антонина Ивановна
Введите дату рождения (dd/mm/yyyy): 20/02/1937
Зарплата: 100000
Семейное положение (женат/замужем/холост): 1
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака.
8) Завершение программы

Введите номер команды: 4
      ФИО Дата рождения Зарплата Женат/замужем
Индекс
1      Иванов Иван Иванович 1990-12-24 24500      True
2      Петров Виктор Степанович 2000-12-12 45000      False
3      Абрамова Ольга Викторовна 2007-04-05 20000      False
4      Фролов Кирилл Петрович 1995-01-01 65000      True
5      Меньшова Антонина Ивановна 1937-02-20 100000      True

Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака.
8) Завершение программы

Введите номер команды:

```

Рис.17 Результат ввода 5 сотрудников

```

Введите номер команды: 2
Введите id:2
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне брака.
8) Завершение программы

Введите номер команды: 4
      ФИО Дата рождения Зарплата Женат/замужем
Индекс
1      Иванов Иван Иванович 1990-12-24 24500      True
3      Абрамова Ольга Викторовна 2007-04-05 20000      False
4      Фролов Кирилл Петрович 1995-01-01 65000      True
5      Меньшова Антонина Ивановна 1937-02-20 100000      True

```

Рис.18 Удаление сотрудника №2

```

Введите номер команды: 4
                                ФИО Дата рождения Зарплата Женат/замужем
Индекс
1      Иванов Иван Иванович      1990-12-24      24500      True
3      Абрамова Ольга Викторовна  2007-04-05      20000      False
4      Фролов Кирилл Петрович    1995-01-01      65000      True
5      Меньшова Антонина Ивановна 1937-02-20      100000     True

Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне бра
ка.
8) Завершение программы

Введите номер команды: 3
Введите id:1
Введите новые данные; для сохранения текущих значений просто нажмите Enter
Введите ФИО сотрудника: Гаврилов Игорь Олегович
Введите дату рождения (dd/mm/yyyy): 30/9/1999
Зарплата: 26500
Семейное положение (женат/замужем/холост): 0
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне бра
ка.
8) Завершение программы

Введите номер команды: 4
                                ФИО Дата рождения Зарплата Женат/замужем
Индекс
1      Гаврилов Игорь Олегович    1999-09-30      26500      False
3      Абрамова Ольга Викторовна  2007-04-05      20000      False
4      Фролов Кирилл Петрович    1995-01-01      65000      True
5      Меньшова Антонина Ивановна 1937-02-20      100000     True

```

Рис.19 Обновление данных у сотрудника №1

```

Введите номер команды: 4
                                ФИО Дата рождения Зарплата Женат/замужем
Индекс
1      Гаврилов Игорь Олегович    1999-09-30      26500      False
3      Абрамова Ольга Викторовна  2007-04-05      20000      False
4      Фролов Кирилл Петрович    1995-01-01      65000      True
5      Меньшова Антонина Ивановна 1937-02-20      100000     True

Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне бра
ка.
8) Завершение программы

Введите номер команды: 3
Введите id:3
Введите новые данные; для сохранения текущих значений просто нажмите Enter
Введите ФИО сотрудника:
Введите дату рождения (dd/mm/yyyy):
Зарплата: 44500
Семейное положение (женат/замужем/холост):
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне бра
ка.
8) Завершение программы

Введите номер команды: 4
                                ФИО Дата рождения Зарплата Женат/замужем
Индекс
1      Гаврилов Игорь Олегович    1999-09-30      26500      False
3      Абрамова Ольга Викторовна  2007-04-05      44500      False
4      Фролов Кирилл Петрович    1995-01-01      65000      True
5      Меньшова Антонина Ивановна 1937-02-20      100000     True

```

Рис.20 Частичное обновление данных у сотрудника №3

```

Введите номер команды: 4
                                ФИО Дата рождения Зарплата Женат/замужем
Индекс
1      Гаврилов Игорь Олегович      1999-09-30      26500      False
3      Абрамова Ольга Викторовна    2007-04-05      44500      False
4      Фролов Кирилл Петрович      1995-01-01      65000      True
5      Меньшова Антонина Ивановна   1937-02-20      100000     True
Выберите действие из списка:
1) Добавление данных о сотруднике
2) Удаление данных о сотруднике
3) Изменение данных о сотруднике
4) Вывод списка сотрудников
5) Вывод списка сотрудников младше заданного возраста
6) Вывод диаграммы распределения зарплат
7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне бра
ка.
8) Завершение программы

Введите номер команды: 5
Введите возраст27
                                ФИО Дата рождения Зарплата Женат/замужем
Индекс
3      Абрамова Ольга Викторовна    2007-04-05      44500      False

```

Рис.21 Вывод списка по условию

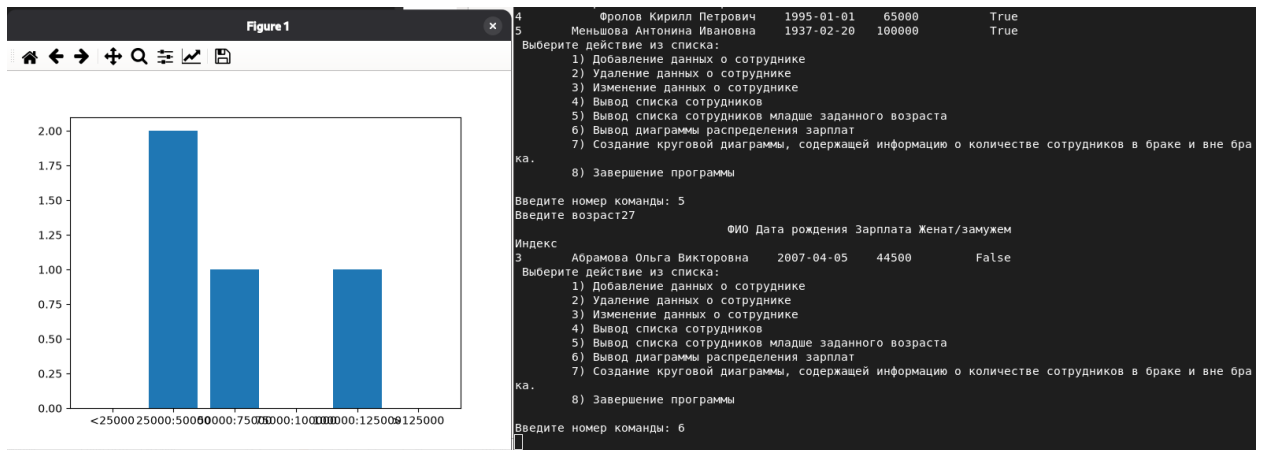


Рис.22 Построение графика №1

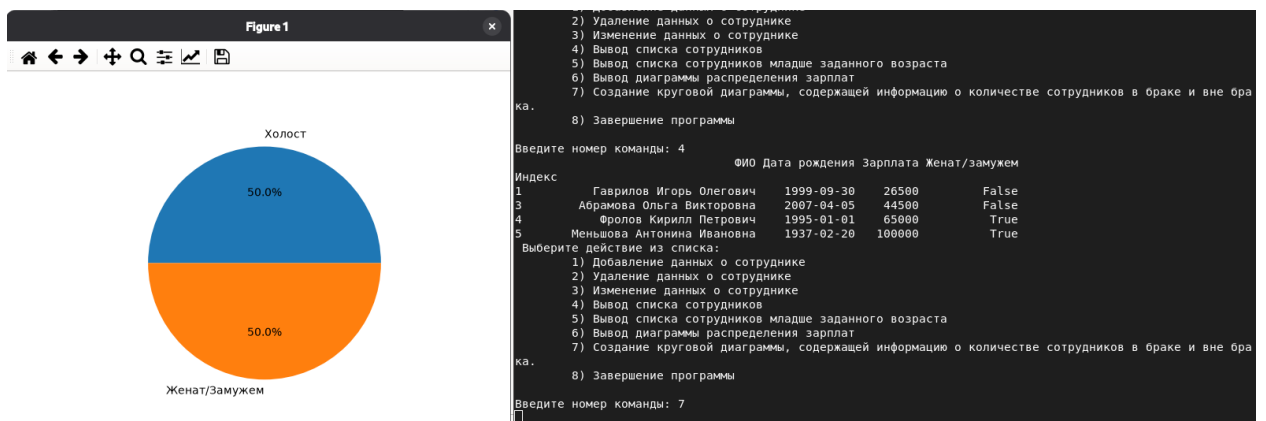


Рис.23 Построение графика №2

```

Введите номер команды: 7
Выберите действие из списка:
  1) Добавление данных о сотруднике
  2) Удаление данных о сотруднике
  3) Изменение данных о сотруднике
  4) Вывод списка сотрудников
  5) Вывод списка сотрудников младше заданного возраста
  6) Вывод диаграммы распределения зарплат
  7) Создание круговой диаграммы, содержащей информацию о количестве сотрудников в браке и вне бра
ка.
  8) Завершение программы

Введите номер команды: 8
(venv) user@debian:~/Документы/ВТм-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ deactivate
user@debian:~/Документы/ВТм-126/Дисциплины/1_семестр/Инжиниринг данных/labs/Lab_1$ █

```

Рис.24 Завершение программы

Дополнительно, исходный код был сохранен в GitHub в репозитории https://github.com/goshagavriloff/engineering_labs/tree/master/Lab_1

ВЫВОД

В ходе выполнения работы были успешно отработаны навыки использования библиотек *NumPy*, *Pandas* и *Matplotlib*.